Learning to Compose

Recursive Self-Improvement of Shared Executable Stacks for Zero-Shot Long-Horizon Robot Manipulation

Yangzhi Chen[1] **Feiran Liu**[1] **Zishuo Wang**[1] **Zijian Jin**[2] **Xin Zhu**[3] **Lecheng Ruan**[4] **Wenjuan Han**[1,5]

[1]Beijing Jiaotong University [2]New York University [3]University of Southern California
[4]PKU-HEC joint lab for industrial intelligence, Peking University [5]LingXing Tech

GitHub: https://github.com/cocacola-lab/Recursia

Project Page: https://cocacola-lab.github.io/Recursia-website/

ABSTRACT

Individually competent robot skills can fail when composed: tools may lack a required capability, completion signals may misrepresent physical progress, and one skill may leave a state that violates its successor's prerequisites. We introduce Recursia, a recursive self-improvement framework that treats the shared executable stack, rather than individual task programs, as the persistent unit of improvement. The stack integrates tools, skills with execution contracts, handoffs, and verifiers. Source-side execution traces guide failure attribution and the generation of missing tools or repairs to existing shared components. Generality checks, mechanism tests, and paired selection and regression gates determine which changes become the next parent. The resulting stack is frozen and reused through declarative task adapters, allowing source-derived changes to support new compositions without target-side code repair. Experiments cover LIBERO-Pro Long simulation and a physical Franka FR3 equipped with a Franka Hand. In simulation, Recursia improves success over ASPIRE by 5.8 percentage points under *Pos* perturbations and 15.1 points under *Task* perturbations. Across three physical compositional target tasks, it improves mean success over the Skill Document Only baseline by 40 percentage points. Analyses of constituent execution, candidate rejection, and inference drivers distinguish local mechanism improvements from complete task success and characterize the cost of feedback-conditioned orchestration. These results examine shared executable interfaces as a concrete mechanism for compositional transfer in robot programming.

1 Introduction

Long-horizon robot manipulation demands more than a repertoire of individually capable skills (Mees et al., 2021). Executing a pick-and-place action, for instance, is insufficient if it leaves the environment in a state that violates the preconditions of the subsequent skill (Xu et al., 2026). Grounding high-level language plans in skill affordances and perceptual geometry is therefore critical for physically feasible execution (Ahn et al., 2022; Huang et al., 2023). The central question is how skills acquired on source tasks can be composed to reliably solve previously unseen long-horizon target tasks.

Coding agents provide a natural testbed for studying this compositional generalization (Liang et al., 2023; Mu et al., 2024; Fu et al., 2026). Language-model-generated programs can invoke perception and control APIs, implement feedback loops, and reuse procedural abstractions (Liang et al., 2023). RoboCodeX further decomposes multimodal instructions into object-centric manipulation units and synthesizes control

code (Mu et al., 2024), while ProgPrompt incorporates state-feedback assertions and recovery actions into generated plans (Singh et al., 2023). Skill-library systems such as Voyager store executable procedures and use environmental feedback for extension (Wang et al., 2023); ASPIRE combines execution feedback, program refinement, skill discovery, and evolutionary search for manipulation (Lu et al., 2026). Lifelong Robot Library Learning expands skill repertoires from accumulated experience (Tziafas and Kasaei, 2024), and Uni-Skill requests new skill implementations when existing ones fall short (Xie et al., 2026). CaP-X examines how abstraction levels, perceptual grounding, and structured execution feedback affect the reliability of embodied coding agents (Fu et al., 2026). Although these capabilities make program improvement a promising route to transfer, reliable composition depends not only on the generated task program, but critically on the *shared interfaces* it invokes: tools, skill contracts, inter-skill state transfers, and verification logic. Thus, the central object of study shifts from *how* an agent writes a program to *which reusable execution semantics* that program inherits.

We identify four key obstacles to reliable composition: (1) **Tool insufficiency**, where a task requires a reusable capability absent from the initial API; (2) **Semantic misalignment**, where an existing tool returns normally despite failing to reach its requested pose of the next tool in the skill; (3) **State incompatibility**, where a skill's terminal state violates its successor's preconditions (Xu et al., 2026); and (4) **Verification brittleness**, where incorrect checks advance an unfinished task or disturb a completed goal. These failure modes correspond to different interfaces in a hierarchical controller: tool execution, skill completion, inter-skill state transfer, and verifiers. Prior systems acquire and reuse executable skills from experience (Wang et al., 2023; Tziafas and Kasaei, 2024; Lu et al., 2026), and execution-guided self-debugging demonstrates that failed programs yield useful repair evidence (Chen et al., 2024). The Darwin Gödel Machine extends this to persistent self-modification evaluated on coding benchmarks (Zhang et al., 2025). We investigate how repairs to shared interfaces with multiple layers can be validated and retained across tasks. We hypothesize that a repair at a shared interface should improve multiple compositions, whereas a task-specific patch should benefit only the trajectory in which it is written. We examine this hypothesis through simulation and physical-robot experiments on long-horizon compositions.

We propose RECURSIA, a framework that targets compositional generalization by treating the *shared executable stack* as the primary object of persistent improvement. A coding agent can add missing tools, repair skill contracts, or improve handoffs and verifiers. Each modification becomes part of a versioned implementation reused by subsequent source tasks and frozen during target-task execution. Specifically, RECURSIA is stack-aware and validation-gated. Source-side execution traces identify the first violated requirement, attributing failures to specific repair layers. This attribution proposes a mechanism for change but does not itself establish efficacy. Candidates must satisfy restrictions on task-specific specialization, survive paired comparisons with the parent stack, and pass the validation gate with selection gate and regression gate. Accepted changes are retained with their failure class, validation evidence, and code version. Task variation is expressed through declarative adapters. At deployment, the frozen stack relies solely on permitted robot observations; evaluator-side ground truth used during source diagnosis is never an input to the deployed policy.

Our contributions are threefold: (1) We formulate skill composition as the persistent improvement of a *shared executable stack* integrating tools, skills with execution contracts, handoffs, and verifiers, and enable the generation of previously absent tools. (2) We introduce a failure-attributed improvement procedure that localizes repairs to specific stack layers and validates shared patches through generality and regression gates. (3) We evaluate frozen-stack transfer to long-horizon compositions in LIBERO-Pro Long simulation and on a physical Franka FR3 robot, comparing with skill-library and skill-document baselines and analyzing compositional execution, validation decisions, and inference strategies.

2 Method

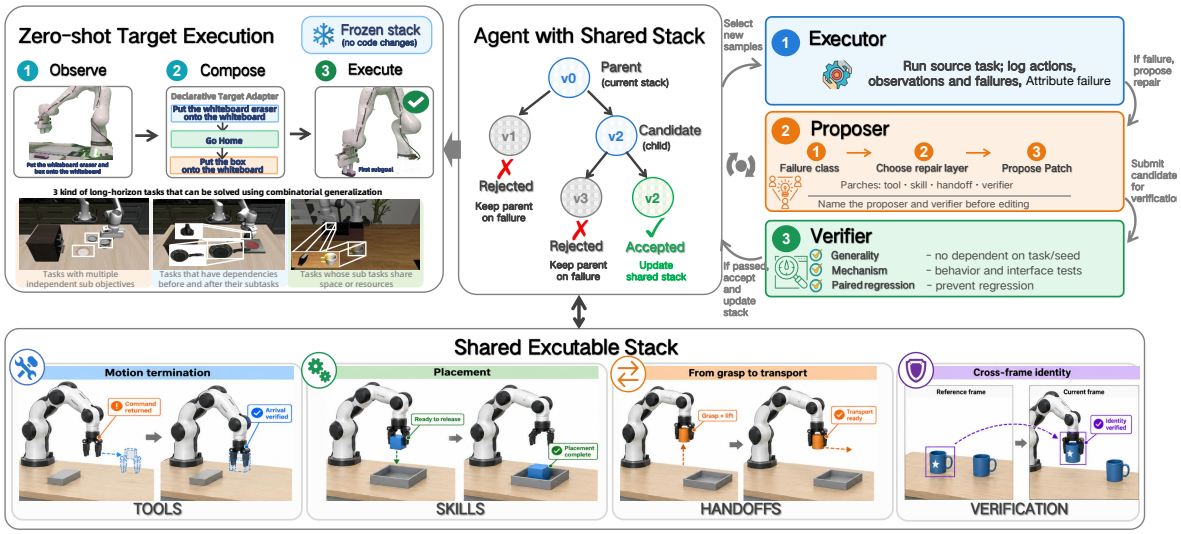


Figure 1: Framework overview. The improvement loop operates on four executable levels. Both newly generated tools and repairs to existing components must pass the same acceptance process before entering the shared stack. Target execution does not modify the stack.

2.1 Problem Formulation

Let $\mathcal{D}_{\text{src}}$ denote the source dataset available for improvement and $\mathcal{D}_{\text{tgt}}$ the target dataset used for frozen transfer. τ denotes one task. The goal is to obtain an executable shared stack through debugging with $\mathcal{D}_{\text{src}}$.

After k accepted modifications obtained from $\mathcal{D}_{\text{src}}$, the executable stack is

$$\mathcal{S}_k = (\mathcal{T}_k, \mathcal{K}_k, \mathcal{H}_k, \mathcal{V}_k), \tag{1}$$

where $\mathcal{T}_k$ contains tools, $\mathcal{K}_k$ reusable skills, $\mathcal{H}_k$ handoff mechanisms, and $\mathcal{V}_k$ verifier mechanisms. Learning and debugging occurs through changes and modifications to executable code; the foundation model is held fixed within a comparison. The declarative adapter a_τ provides object descriptions, support or destination descriptions, relation types, and goal dependencies. A shared executor composes calls to these components using a_τ and the available observation history $h_t = (o_0, u_0, \ldots, o_t)$.

We use *zero-shot compositional transfer* to mean that target-task compositions do not provide feedback for stack modification, and the frozen stack is executed without target-specific code repair. Source and target tasks may share objects, relations, or individual skills.

2.2 Four-layer Shared Executable Stack

Tool. Tools expose reusable perception, geometry, planning, or execution operations. Their interfaces specify inputs, coordinate conventions, observable completion conditions, and resource limits. The tool set is extensible. When an attributed source failure reveals that no available operation can satisfy a required capability, the agent proposes a new shared tool rather than embedding a task-specific workaround in the adapter. A proposal defines the missing capability, an interface, the permitted observations it consumes, and tests of its intended behavior. New tools are assembled from the permitted lower-level operations and are registered only after validation. Tool generation cannot introduce privileged sensing at deployment or alter the external evaluator. We distinguish these additions from repairs to existing tools in the experimental analysis. Figure 2 illustrates representative interfaces.

Skill. A skill with execution contracts is represented by

$$K_i = (\text{Pre}_i, \pi_i, \text{Post}_i, B_i), \tag{2}$$

where Pre_i defines observable prerequisites, π_i is a parameterized program, Post_i specifies evidence required at completion, and B_i bounds resource use. Preconditions and postconditions refer to task arguments and measured geometry, not identities of benchmark tasks. Execution returns evidence and a reason for termination, allowing the caller to distinguish a completed motion from an established task relation. Contracts are executable checks, not formal guarantees of correctness in unobserved states.

Handoff. The completion of one skill does not imply readiness for the next:

$$\mathrm{Post}_i(h_t) \not\Rightarrow \mathrm{Pre}_{i+1}(h_t). \tag{3}$$

A handoff updates the state used by the successor, checks its prerequisites, and checks previously achieved relations that later actions could disturb. Depending on the observed failure, the shared executor may re-observe, invoke a bounded recovery, or stop. Recovery must establish the missing prerequisite before execution resumes. For instance, a completed release motion requires a subsequent retreat before the next grasp becomes executable. The handoff makes this transition explicit.

Verifier. The verification layer evaluates object identity and task-relevant relations from permitted observations and execution history. The external evaluator independently measures its errors; the internal evaluator's own output is never used as the ground-truth success label. The external evaluator's role is distinct from a motion tool's arrival check: reaching a pose does not establish that the intended object is supported by the intended surface. The external verification uses support geometry and object identity across observations to decide whether the evidence permits advancement or recovery.

Together, these levels define the unit of reuse. A validated tool can serve several skills, a skill can appear in several goal sequences, and a handoff or relation check can apply across tasks.

Figure 2 shows four representative shared interfaces. The layers group responsibilities within one executable stack.

2.3 Failure-Attributed Improvement Procedure

The executor run source rollouts and records actions, observations and robot state. Source diagnosis use simulator ground truth to distinguish failure mechanisms, while the deployed stack remains restricted to permitted observations. The attribution procedure identifies the earliest supported violation of an execution requirement and records its stage, evidence, and affected component. It distinguishes an absent capability from an incorrectly implemented one.

The coding agent receives this evidence and proposes a candidate modification δ. The proposal names the failure class, the stack level to modify, the intended repair, and the tests that would evaluate the repair. Examples in Figure 2 include adding a missing constrained-motion operation, preventing unexecuted planning candidates from changing later solves, or revising a handoff that starts its successor before transport has completed. The candidate repair modify an existing component of $\mathcal{S}_k = (\mathcal{T}_k, \mathcal{K}_k, \mathcal{H}_k, \mathcal{V}_k)$ or extend the inventory. For instance, the candidate repair can extend the tool inventory as follows:

$$\mathcal{S}' = \mathrm{Apply}(\mathcal{S}_k, \delta), \qquad \mathcal{T}' = \mathcal{T}_k \cup \{t_{\mathrm{new}}\} \quad \text{for an accepted tool addition.} \tag{4}$$

At each improvement iteration, the agent selects one shared-module slot for modification. Candidate changes are confined to that slot. For example, when the shared-tool slot is selected, the other shared components, diagnostic procedures, retrieval, search code, and data splits remain fixed during that iteration. First-failure attribution guides where to intervene, while the mechanism tests and paired execution determine whether that intervention works.

RECURSIA also allows the strategy update to change across iterations. Specifically, adjustments to the verification gate and the types of data used for debugging alter how candidate modifications are

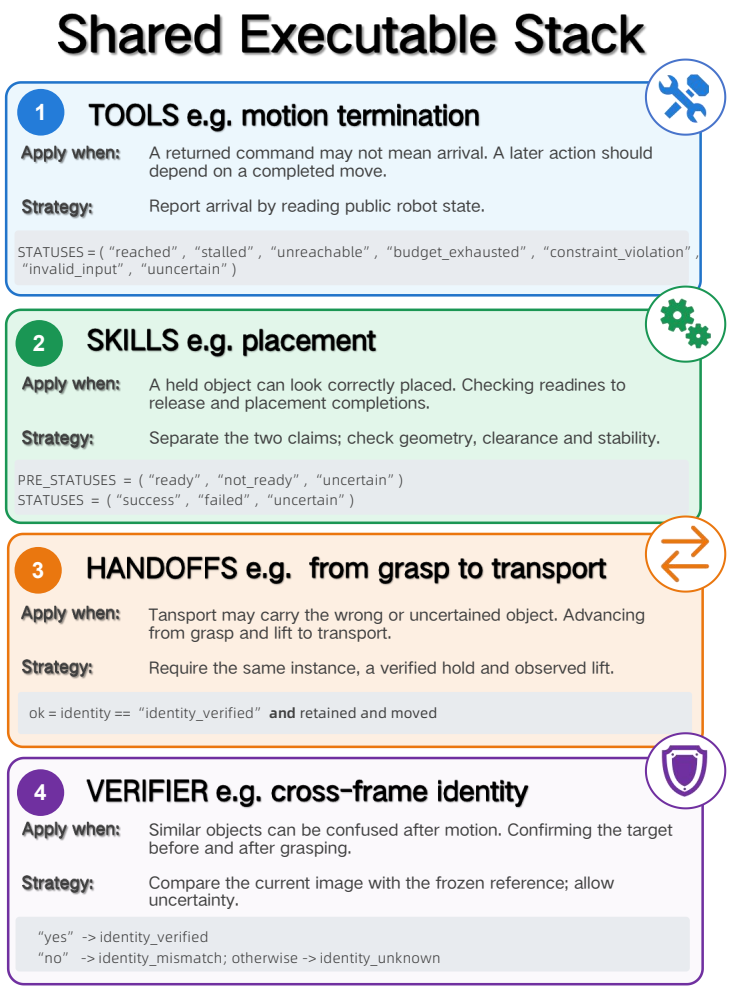


Figure 2: Representative shared-stack interfaces.

evaluated and which failures guide subsequent repair. These changes persist into later iterations, making the improvement procedure itself an object of adaptation.

2.4 Multidimensional Validation Gate

Candidate acceptance combines three types of evidence. Firstly, generality checks reject benchmark-specific specialization in shared code, including task or seed identifiers and object- or scene-specific control branches. Geometric constants require a documented source, such as robot calibration, measured geometry, or a declared object-independent construction.

Secondly, mechanism tests check the proposed behavior before broader execution. Perception and verification patches can be replayed on recorded observations. Motion patches require execution tests, because an old trajectory does not reveal what a new controller would do. Tests also check interface compatibility and whether downstream actions respect failure returns. A newly generated tool must demonstrate both its intended capability and bounded handling of unsupported requests.

Finally, parent and candidate are compared on identical source-task states under a resource policy fixed before candidate evaluation in the current iteration. Let $G(\delta)$ indicate the generality checks, $U(\delta)$ the mechanism tests, and $R(\mathcal{S}', \mathcal{S}_k; \mathcal{D}_{\text{sel}}, B)$ the paired performance and regression gate. Acceptance is

$$\text{Accept}(\delta) = G(\delta) \wedge U(\delta) \wedge R(\mathcal{S}', \mathcal{S}_k; \mathcal{D}_{\text{sel}}, B). \tag{5}$$

The gate considers source-task success, regressions on other source tasks, and resource use. At the beginning of each RSI iteration, the coordinator fixes the evaluation panels, acceptance criteria, and

Algorithm 1 RECURSIA Debugging Phrase for One Iteration

Require: Initial stack $\mathcal{S}_0$; source development task and selection tasks; budget B

$k \leftarrow 0$
for each source-task improvement step within budget **do**
  Execute $\mathcal{S}_k$ with a source-task adapter
  Attribute the first supported failure from aligned source traces
  Identify a missing capability or a faulty shared component
  Propose δ: generate a tool or repair the selected stack level
  $\mathcal{S}' \leftarrow \mathrm{Apply}(\mathcal{S}_k, \delta)$ in an isolated candidate version
  if generality checks and mechanism tests pass **then**
    Run parent-candidate paired development and selection tests
    if Multidimensional validation gates pass **then**
      $\mathcal{S}_{k+1} \leftarrow \mathcal{S}'$; archive repair evidence; $k \leftarrow k + 1$
    else
      Retain $\mathcal{S}_k$ and archive the rejection
    end if
  end if
end for
Inference: Execute target adapters using the frozen stack without target-driven repair

resource budgets for that iteration. These settings may change between iterations as the stack evolves, but remain fixed when comparing the parent with candidates within the same iteration. Rejected candidates do not replace the parent. Accepted candidates retain their code hash, source failure, tests, and paired results.

At inference time, all shared implementations are frozen. The target adapter specifies the requested composition, and execution logs identify the versions invoked by each subgoal.

3 Experiments

Our experiments examine whether source-side improvement of a shared executable stack transfers to unseen long-horizon compositions and which levels of the shared stack contribute to transfer.

3.1 Experimental Setup

Setup. We use LIBERO-90 (Liu et al., 2023) for debugging, including failure diagnosis, repair generation, and candidate selection. Improvements persist in a versioned executable stack rather than in target-specific programs. All source-side development is completed before target evaluation, and the selected stack is frozen together with its tools, skill implementations, handoff logic, and verifiers. The coding model and its parameters remain fixed within each controlled comparison.

The first track evaluates the frozen stack on LIBERO-Pro Long, following the long-horizon transfer setting studied by ASPIRE (Zhou et al., 2025; Lu et al., 2026), with 10 long-horizon tasks evaluated along two axes. *Pos* changes initial object positions; *Task* changes the task specification. Each task–axis pair is evaluated on seeds 1–50, yielding 500 episodes per axis. The 10 task names and their axis-specific results are listed in Appendix D. The second track evaluates on a physical Franka FR3 equipped with a Franka Hand. Physical evaluation trajectories are not used for stack repair or selection.

Baselines. We compare with OpenVLA (Kim et al., 2024), π_0 (Black et al., 2025), $\pi_{0.5}$ (Physical Intelligence et al., 2025), CaP-Agent0 (Fu et al., 2026), and ASPIRE (Lu et al., 2026). CaP-Agent0's published protocol regenerates programs per seed with test-time reasoning and retries, whereas ASPIRE

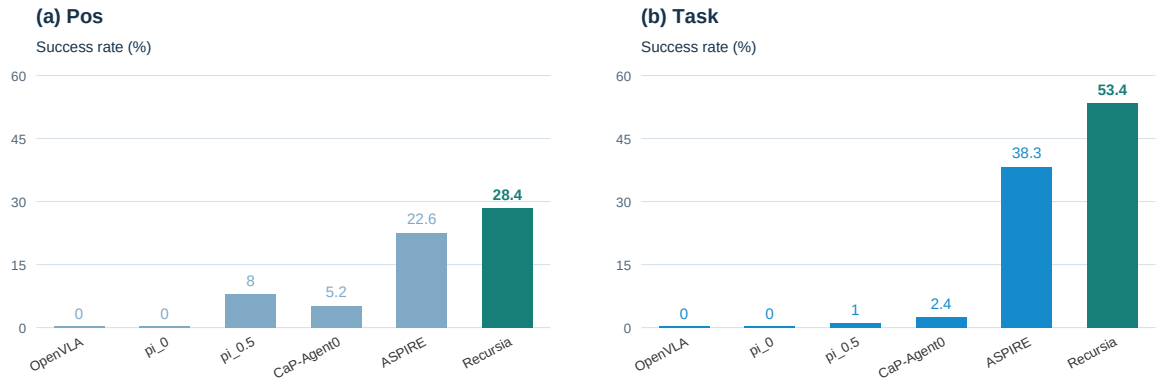


Figure 3: Zero-shot transfer to LIBERO-Pro Long. (a) Position perturbations (*Pos*). (b) Task-specification perturbations (*Task*). Baseline success rates are reproduced from the existing comparison; RECURSIA achieves success rates of 0.284 on *Pos* and 0.534 on *Task*. Aggregate and per-task tables appear in Appendix D.

fixes one program per task without target debugging; this difference is retained when interpreting the comparison.

Metrics. We measure full-task success using the official benchmark evaluator. For task i and perturbation axis a, success is the fraction of successful episodes among its 50 evaluation states. Axis success is the unweighted mean over the 10 task rates; Overall is the mean of the *Pos* and *Task* macro-averages. Rates are reported on $[0, 1]$. Failures, timeouts, and program errors remain in the denominator. We report source-improvement costs separately from target-execution costs, including model input/output tokens, wall time, and perception calls. Matched reruns use identical initial states and report paired outcomes.

3.2 Evaluation in Simulation: Zero-Shot Transfer to Long-Horizon Tasks on LIBERO-Pro-Long

Here N denotes the number of improvement iterations; each iteration processes five source samples. Figure 3 presents the primary zero-shot comparison separately for position (*Pos*) and task-specification (*Task*) perturbations. The source-experience scaling analysis is provided in Appendix D.2, Figure 7. Exact aggregate values are provided in Appendix D, Table **??**. We report separately frozen RECURSIA stacks after $N \in \{0, 3, 6, 9, 10, 12, 13, 14, 15, 16, 17, 18\}$ improvement iterations to examine how transfer changes with source-side experience. Each snapshot is selected using source data only and evaluated without target-driven modification. This comparison tests whether source-side changes to shared execution mechanisms remain useful when the target composition changes and no target feedback is available for repair. RECURSIA achieves success rates of 0.284 on *Pos* and 0.534 on *Task*. Across the supplied scaling checkpoints, success rises from 0 at $N = 0$ to 0.232 and 0.434 at $N = 18$, respectively, with nonmonotonic intermediate changes.

3.3 Evaluation on a Physical Robot: Zero-Shot Transfer to Long-Horizon Tasks with Franka FR3

We compare RECURSIA with Skill Document Only (SD-only) on a Franka FR3 equipped with a Franka Hand. SD-only uses the tool implementation of ASPIRE and updates a single natural-language skill document, whereas RECURSIA updates a shared executable Python stack. Both use GPT-6 Astra, front and wrist observations, and the same six robot tools with SAM3 and Contact-GraspNet.

The six physical source tasks cover placing a mug or milk carton in a basket, placing white and blue mugs on the left and right plates, placing a plastic bowl on a plate, and placing a mug in the bowl. Each source task contributes three retained development rollouts. SD-only runs three passes through the source suite; RECURSIA groups the tasks into three two-task chunks, evaluating the current stack and two candidate

Table 1: Success rates (%) on physical compositional target tasks using Franka FR3 with Franka Hand.

Task	SD-only	RECURSIA
T1: Shared basket	0.0	20.0
T2: Reassigned plates	60.0	80.0
T3: Bowl–mug dependency	20.0	100.0
Average	26.7	66.7

updates on each pair.

The targets compose these actions: T1 places the mug and milk carton in the same basket; T2 places the blue mug on the left plate and the white mug on the right plate, reversing their source assignments; T3 first places the bowl on the plate, then the mug in the relocated bowl. Source knowledge is frozen before evaluation. Within-trial feedback corrections do not update persistent knowledge. Each target is evaluated ten times per method, with at most 20 code blocks per trial. Task success is assessed manually.

3.4 Analysis of Compositional Generalization

The RECURSIA framework targets compositional generalization, combining knowledge from multiple source tasks to enable zero-shot transfer to long-horizon target tasks. This subsection aims to demonstrate this capability through retrospective case studies. Compositional transfer requires both competent constituent skills and compatible states at their boundaries. To disentangle improvements in individual skills from those in compositional ability, we examine this distinction through retrospective execution cases, separating two events: completion of the predecessor's goal and establishment of the successor's prerequisites. In particular, while the predecessor task is complete in the current state, this does not guarantee that the next object can be readily identified, reached, or manipulated. A useful diagnostic is the difference between isolated and chained success:

$$\Delta_{\text{chain}} = p_{\text{success}}^{\text{isolated}} - p_{\text{success}}^{\text{chained}} \tag{6}$$

This quantity measures sensitivity induced by composition capability.

Improving isolated skills does not remove the downstream bottleneck. At $N = 11$, the standalone stove-on operation improved from $0/10$ to $10/10$ on both development and selection datasets, whereas the stove-pan composition remained $0/10$. In the inspected development rollouts, the stove-on predicate became *True*, but subsequent manipulation lost the pan after a grasp attempt; alternative candidates were also rejected as outside the inverse-kinematics workspace. Thus, the rotary repair improved the first operation without establishing the conditions needed to complete the second. This comparison locates the remaining bottleneck and shows the difference between isolated and chained success. Our approach filled this gap by handoff repair mechanism, in the later iterations.

Relational success and successor readiness are distinct. In the $N = 6$ mug-pudding sequence, the mug-on-plate predicate became *True* in all ten *Pos* episodes, while the full conjunction succeeded in only one. In seed 42, the mug goal became *True* at step 573 and remained *True*, but subsequent pudding binding returned `not_found`. Seed 41 provides a successful contrast: the mug goal became *True* at step 552 and the pudding goal at step 1440, with the first goal preserved (Figure 4). A drawer sequence exhibits the same failure structure: seed 48 achieved bowl containment, but closing stopped at `part_not_found`. These traces identify perceptual binding as a prerequisite that is not certified by predecessor goal completion.

Together, these cases distinguish failures of successor execution from failures to establish successor readiness. The observed mug-pudding failure belongs to the second category, with the first goal preserved; the stove-pan failure exposes downstream manipulation limitations after stove-on succeeds. These evidence supports a conclusion: predecessor completion and successor readiness must be examined separately and

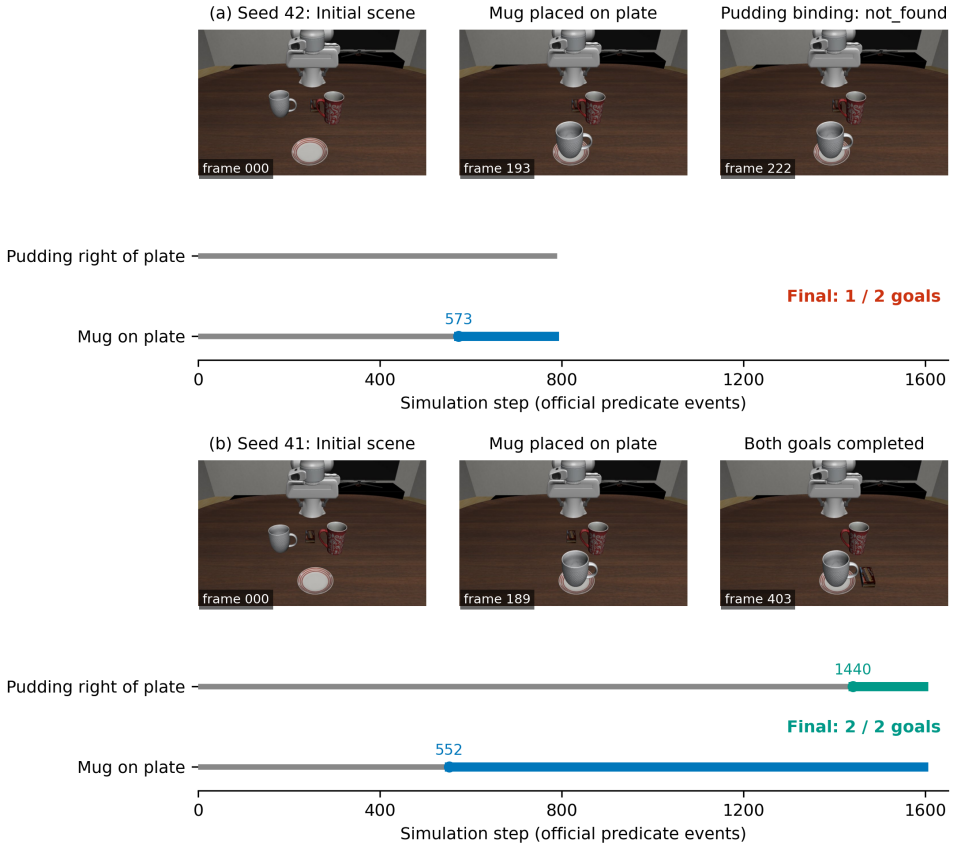


Figure 4: Recorded compositional generalization cases. Original RGB recordings and official predicate timelines for the N=6 mug–pudding sequence. (a) Seed 42 completes the mug goal, but pudding binding returns `not_found`; the earlier goal remains true through termination. (b) Seed 41 completes both goals while preserving the first. Gray segments denote false predicates; blue and teal segments denote true predicates. Across ten swap-suite episodes, the mug predicate is true at termination in all ten, whereas the full conjunction succeeds in one. The two illustrated seeds have different initial states. RGB frame identifiers and simulation-step timestamps use separate counters.

processed by the handoff mechanism.

3.5 Analysis of Inference Methods

All inference methods are driven by declarative adapters. We compare three inference drivers over the same frozen shared stack: rule-based driver, one-shot task-program driver, and online closed-loop driver. These drivers differ in how high-level execution decisions are specified and updated. The rule-based driver follows predefined decision rules. The one-shot task-program driver executes an initially generated program, which may include observation-dependent branches and bounded retries. The online closed-loop[1] driver conditions subsequent decisions on observations and execution outcomes throughout the episode. All three may use observation feedback and closed-loop low-level control. The results are shown in Figure 5. One-shot generation describes the lifetime of the program, rather than an absence of feedback during its execution. In ASPIRE's zero-shot setting, task-program generation excludes target-side debugging, but the one-shot generated program may still re-observe the scene and use closed-loop low-level control (Lu et al., 2026). Here, we use the frozen shared stack.

The online closed-loop driver outperforms the one-shot task-program driver. The comparison between the online closed-loop driver and one-shot task-program driver is more limited. After multiple

[1]The label "closed-loop" here denotes the inference driver's decision process, rather than a property exclusive to its low-level robot controller.

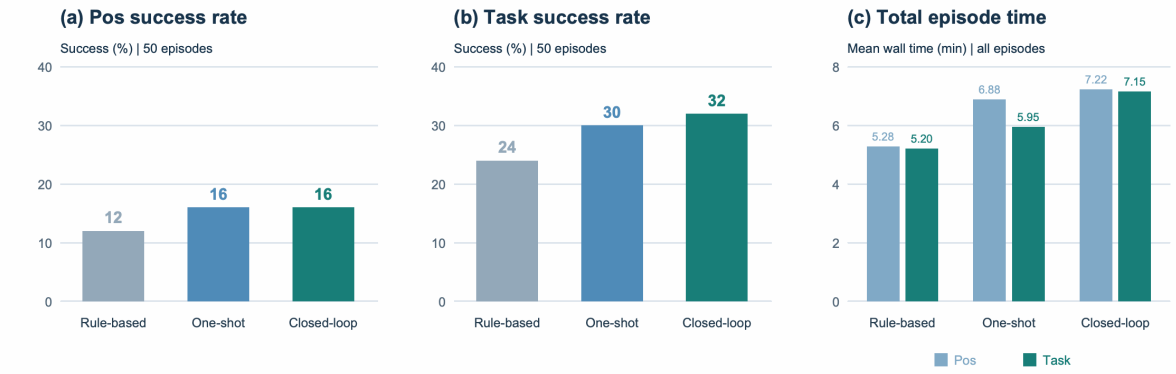


Figure 5: Inference-driver comparison on a frozen stack of the $N = 13$ iteration. (a) *Pos* and (b) *Task* success rates, each 50 matched episodes. (c) Mean end-to-end episode time, including program generation and robot execution, for all episodes.

tries, the online closed-loop driver always gains more successful trajectories. The trajectories show that the drivers have different failure and success profiles. It reveals that the closed-loop decision making may resolve failures encountered by a fixed generated program while introducing other failures.

Feedback incurs latency without uniform gains across perturbation axes. The rule-based, one-shot, and closed-loop drivers achieve *Pos* success rates of 12%, 16%, and 16%, and *Task* success rates of 24%, 30%, and 32%, respectively. Mean end-to-end episode times are 5.28, 6.88, and 7.22 minutes on *Pos*, and 5.20, 5.95, and 7.15 minutes on *Task* (Figure 5). These durations include generation and execution, with runtime model latency counted within execution; failures and timeouts remain in the averages.

Feedback-condition mechanism incurs latency without uniform gains across perturbation axes. Mean end-to-end episode times are 5.28, 6.88, and 7.22 minutes on *Pos*, and 5.20, 5.95, and 7.15 minutes on *Task*. These durations include generation and execution, with runtime model latency counted within execution; failures and timeouts remain in the averages. Relative to one-shot generation, closed-loop inference adds approximately 21 seconds per *Pos* episode, and 72 seconds per *Task* episode for a two-percentage-point gain. The incremental benefit is therefore concentrated in *Task* perturbations, whereas the *Pos* results do not show an aggregate improvement from additional feedback-conditioned decisions. Relative to the rule-based driver, one-shot generation gains four and six percentage points at additional mean costs of approximately 96 and 45 seconds on *Pos* and *Task*. In this comparison, much of the observed success improvement is already obtained by one-shot orchestration.

3.6 Analysis of the Validation Gate

The RECURSIA framework incorporates two critical validation components for the validation gate, i.e., the *selection gate* and the *regression gate*, to address two distinct risks: a candidate may correct a local mechanism without improving overall task completion (mitigated by the selection gate), or improve aggregate completion while disrupting previously successful states (mitigated by the regression gate). This subsection uses retrospective case studies to demonstrate their effectiveness. In the book-transfer task, the development summary reported candidate-filter rejection in eight of ten episodes for early iterations $N = 8$. Inspecting the earlier execution revealed a defect: the robot had already grasped and lifted the intended book, but a post-lift visual query selected a similar book that remained on the table. The low re-detected position was treated as evidence of a failed lift, causing the gripper to open and subsequent grasp attempts to fail. The subsequent iterations attempt to repair this identity check. We employ the validation framework to ensure the recognition of a known stationary competitor following a verified lift, while preserving the existing motion and hold checks.

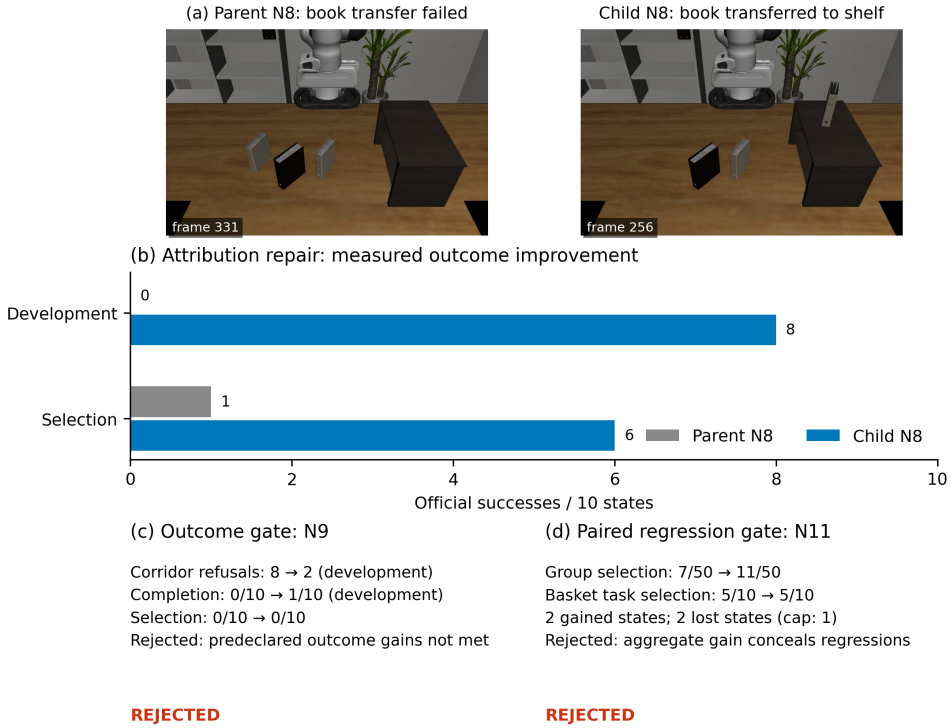


Figure 6: Recorded validation-gate cases. (a) Initial state. Original simulator frames from the N=8 iteration, both with seed 86. (b) Development and selection successes for this task. (c) The iteration N=9 reduced a local failure mechanism but failed the predeclared outcome gains. (d) The iteration N=11 improved the aggregate selection score while losing two previously successful basket-placement states, exceeding the per-task loss cap.

Selection gate rejects candidates with insufficient improvement on the selection dataset. A repair during $N = 9$ changed the transport-to-descent boundary for book insertion so that entry into the opening remained under contact-aware control. Development release refusals attributed to the lateral corridor decreased from 8 to 2, and most tests passed. Nevertheless, official completion improved only from $0/10$ to $1/10$ in the development dataset and remained $0/10$ in the selection dataset, below the predeclared gains of two and one successes, respectively. The reason is that removing a prominent failure mechanism exposed remaining arrival, contact, and grasp failures. Although the candidate passed the paired-loss checks, it was rejected by the selection gate for insufficient improvement on the selection dataset.

Regression gate prevents aggregate gains from concealing lost successes. The N=11 iteration increased development completion from $2/50$ to $3/50$ and selection completion from $7/50$ to $11/50$. However, one basket-placement task retained its selection total of $5/10$ by gaining two states and losing two previously successful states. The two losses exceeded the per-task cap of one, and the candidate was rejected by the regression gate. Development book-insertion corridor refusals also fell from nine to zero without producing a completed insertion. These cases motivate checking trajectories, outcomes and paired losses in addition to aggregate development scores (Figure 6).

3.7 Analysis of Tool Repair and Tool Generation

Table 2 summarizes representative entries and shows generated repairs extend the callable inventory. The N=10 iteration added support-instance and placement semantics for previously refused `front/middle` relations, reusing the existing instance selector and placement primitives. Instruction dispatch coverage increased from $81/90$ to $84/90$, and the `front-of-mug` task improved from $0/10$ to $8/10$ in development dataset and $0/10$ to $7/10$ in selection dataset. The N=11 iteration subsequently generated a shared `run_rotary` driver that served both turn-on and turn-off instructions, corrected the pinch frame, and

Entry	Failure	Repair	Recorded Result
N=10	Lack of front/middle reference relations	Shared instance selection and relation dispatch	Dispatch $81/90 \rightarrow 84/90$; gains measured on source tasks.
N=11	Rotary localization, pinch, and arc defects	Shared turn-on/off driver	Turn-on $10/10$ in each panel.

Table 2: Representative repairs. Rates refer to the specified tasks, not the complete Long-Pro benchmark.

re-anchored turn increments on the executed tool pose. Twenty tests passed, and the driver was invoked in both standalone stove activation and the stove–pan sequence.

4 Related Work

Embodied Agents. Language-guided robot agents connect high-level reasoning to physical execution through learned action models or executable programs. Vision-language-action models (VLA) such as RT-2, OpenVLA, and π series map visual observations and language instructions to robot actions (Brohan et al., 2023; Kim et al., 2024; Black et al., 2025). Programmatic approaches instead express control through calls to perception and action interfaces. Code as Policies composes these calls into robot programs and recursively generates missing helper functions (Liang et al., 2023), while ProgPrompt grounds plan generation in programmatic descriptions of available actions and objects (Singh et al., 2023). CaP-X extends this setting with systematic evaluation of coding agents, structured execution feedback, and automatic skill synthesis (Fu et al., 2026). Together, these approaches provide the building blocks for language-guided manipulation. RECURSIA focuses on how those building blocks improve across tasks: it persistently modifies the shared tools, skills, handoffs, and verifiers that generated programs invoke.

Embodied Recursive Self-Improvement. A central question in recursive self-improving agents is where experienc and knowledge is retained and how it changes subsequent behavior. The Darwin Gödel Machine retains improvements in the agent's own code and evaluates successive variants on coding benchmarks (Zhang et al., 2025). In robotics, Eureka uses language models to generate reward functions for policy learning (Ma et al., 2024), while evolutionary code search directly synthesizes executable control policies (Guo et al., 2026). Skill-library approaches retain experience as reusable procedures. Voyager accumulates executable skills in Minecraft (Wang et al., 2023); Lifelong Robot Library Learning bootstraps composable skills for robot control (Tziafas and Kasaei, 2024); and Uni-Skill requests new implementations when its existing repository cannot support a plan (Xie et al., 2026). These approaches make accumulated experience available to later tasks through improved policies, programs, or skill repertoires. ASPIRE (Lu et al., 2026) brings execution-driven repair and skill accumulation into a continual robot-programming loop. It uses execution traces to diagnose failures, distills validated fixes into a transferable skill library. Its evaluation includes transfer to unseen long-horizon tasks, making it the closest precursor to RECURSIA. Zero-shot transfer is also studied in the context of lifelong learning (**?**), where a robot learns to perform new tasks without additional training. RECURSIA differs from these approaches by focusing on the shared executable stack as the unit of improvement, allowing for more systematic and reusable repairs across tasks.

Compositional Generalization for Robotics. Compositional generalization requires familiar capabilities to remain effective when objects, goals, or action sequences are combined in new ways. CLIPort studies language-conditioned manipulation through semantic and spatial abstractions (Shridhar et al., 2021), and VIMA evaluates progressively more novel combinations of task components (Jiang et al., 2022). At longer horizons, CALVIN evaluates sequences of language-conditioned manipulation tasks (Mees et al., 2021), while LIBERO studies knowledge transfer across spatial, object, goal, and task variations (Liu et al., 2023).

LIBERO-PRO adds perturbations to initial object positions and task specifications (Zhou et al., 2025). These evaluation settings expose several dimensions of transfer, from recognizing a new combination of familiar concepts to executing a sequence under changed conditions.

Reliable sequence execution also depends on the states passed between skills. BATON addresses this dependency by exploring subtasks separately and storing transition-aware memory: verification checks readiness, handoff restoration repairs entry conditions, and lookahead favors outcomes that the next subtask can use (Xu et al., 2026). Its treatment of transitions makes explicit why a skill can succeed in isolation yet fail as part of a chain. RECURSIA places this dependency within a broader stack-improvement problem. Handoff repairs are validated alongside changes to tools, skill contracts, and verification, and their shared implementations are reused unchanged on target compositions. Our evaluation therefore examines both the competence of individual skills and the reliability of their composition, tracing transfer to source-side changes in the executable stack.

5 Limitations and Conclusion

Our evaluation has several limitations. First, source-side ground-truth diagnosis provides information that may not be available on a physical robot. In simulation, failed episodes can be reset to a controlled initial state and replayed repeatedly. On a physical robot, resetting the scene can be slow, unsafe, or impossible after an object has moved, and the diagnostic evidence needed to identify the first violated requirement is often incomplete or expensive to obtain. Deployment does not consume simulator ground truth, but transferring the improvement procedure itself to real hardware therefore requires reliable alternatives, such as calibrated state logging, external sensing, human annotations, or conservative uncertainty handling. Second, we have not evaluated cross-embodiment generalization. All source and target experiments use the same robot embodiment and execution interfaces. A shared repair may depend on kinematics, gripper geometry, controller conventions, or calibration details, so transfer across robot platforms remains untested.

We present a framework for improving the shared executable mechanisms through which coding agents compose robot skills. The stack can grow when a required tool is missing and change when existing tools, contracts, handoffs, or verification fail. Source failure attribution directs these changes, while validation gates determine which changes persist. Frozen reuse then connects a source-side modification to the code executed in an unseen long-horizon composition. The primary LIBERO-Pro Long results are 28.4% on *Pos* and 53.4% on *Task*.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gober, Karol Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025. doi: 10.48550/arXiv.2410.24164. URL https://arxiv.org/abs/2410.24164.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models

transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. doi: 10.48550/arXiv.2307.15818. URL https://arxiv.org/abs/2307.15818.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

Hao-Shu Fang, Chenxi Wang, Hongjie Fang, Minghao Gou, Jirong Liu, Hengxu Yan, Wenhai Liu, Yichen Xie, and Cewu Lu. Anygrasp: Robust and efficient grasp perception in spatial and temporal domains. *IEEE Transactions on Robotics*, 39(5):3929–3945, 2023.

Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Guanzhi Wang, Dantong Niu, Fei-Fei Li, Guanya Shi, Jiajun Wu, Shankar Sastry, Yuke Zhu, Ken Goldberg, and Linxi Fan. Cap-x: A framework for benchmarking and improving coding agents for robot manipulation. *arXiv preprint arXiv:2603.22435*, 2026. doi: 10.48550/arXiv.2603.22435. URL https://arxiv.org/abs/2603.22435.

Ping Guo, Chao Li, Yinglan Feng, and Chaoning Zhang. Code evolution for control: Synthesizing policies via LLM-driven evolutionary search. *arXiv preprint arXiv:2601.06845*, 2026. doi: 10.48550/arXiv.2601.06845. URL https://arxiv.org/abs/2601.06845.

Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3D value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023.

Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. VIMA: General robot manipulation with multimodal prompts. *arXiv preprint arXiv:2210.03094*, 2022. doi: 10.48550/arXiv.2210.03094. URL https://arxiv.org/abs/2210.03094.

Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024. doi: 10.48550/arXiv.2406.09246. URL https://arxiv.org/abs/2406.09246.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. doi: 10.48550/arXiv.2209.07753. URL https://arxiv.org/abs/2209.07753.

Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.48550/arXiv.2306.03310. URL https://arxiv.org/abs/2306.03310.

Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi Jim Fan, and Guanzhi Wang. ASPIRE: Agentic /skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026. doi: 10.48550/arXiv.2607.00272. URL https://arxiv.org/abs/2607.00272.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024. doi: 10.48550/arXiv.2310.12931. URL https://arxiv.org/abs/2310.12931.

Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *arXiv preprint arXiv:2112.03227*, 2021. doi: 10.48550/arXiv.2112.03227. URL https://arxiv.org/abs/2112.03227.

Yao Mu, Junting Chen, Qinglong Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, Mengkang Hu, Chaofan Tao, Peize Sun, Haibao Yu, Chao Yang, Wenqi Shao, Wenhai Wang, Jifeng Dai, Yu Qiao, Mingyu Ding, and Ping Luo. Robocodex: Multimodal code generation for robotic behavior synthesis. In *International Conference on Machine Learning (ICML)*, 2024.

Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Conference on Robot Learning (CoRL)*, 2025.

Mohit Shridhar, Lucas Manuelli, and Dieter Fox. CLIPort: What and where pathways for robotic manipulation. *arXiv preprint arXiv:2109.12098*, 2021. doi: 10.48550/arXiv.2109.12098. URL https://arxiv.org/abs/2109.12098.

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. doi: 10.48550/arXiv.2209.11302. URL https://arxiv.org/abs/2209.11302.

Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. Contact-graspnet: Efficient 6-dof grasp generation in cluttered scenes. In *2021 IEEE international conference on robotics and automation (ICRA)*, pages 13438–13444. IEEE, 2021.

Georgios Tziafas and Hamidreza Kasaei. Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. doi: 10.48550/arXiv.2406.18746. URL https://arxiv.org/abs/2406.18746.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.48550/arXiv.2305.16291. URL https://arxiv.org/abs/2305.16291.

Senwei Xie, Yuntian Zhang, Ruiping Wang, and Xilin Chen. Uni-skill: Building self-evolving skill repository for generalizable robotic manipulation. *arXiv preprint arXiv:2603.02623*, 2026. doi: 10.48550/arXiv.2603.02623. URL https://arxiv.org/abs/2603.02623.

Bingxin Xu, Yuzhang Shang, and Emilio Ferrara. Don't drop the BATON: Long-horizon robot manipulation via agentic subtask exploration and transition-aware memory. *arXiv preprint arXiv:2608.16889*, 2026. doi: 10.48550/arXiv.2608.16889. URL https://arxiv.org/abs/2608.16889.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025. doi: 10.48550/arXiv.2505.22954. URL https://arxiv.org/abs/2505.22954.

Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025. doi: 10.48550/arXiv.2510.03827. URL https://arxiv.org/abs/2510.03827.

A Debugging Pipeline

This appendix documents the partial artifact, including the implementation description. The debugging pipeline separates proposal generation from evaluation. The verifier runs a frozen parent, records public observations and simulator/physical diagnostics, and supplies development evidence to one proposer. The proposer diagnoses a failure, makes several proposals, modifies shared code, replays the proposed decisions offline, and supplies tests. The verifier builds only after the final report, checks the candidate, runs a fixed paired plan, and records acceptance or rejection. An accepted program becomes the next parent; a rejected candidate leaves the parent unchanged. The prompt boxes reproduce templates and partial excerpts. Deployment-specific paths and login fields are replaced by `$ASPIRE_ROOT`, `<user>`, and `<server>`; typographic punctuation is normalized. Braced variables are filled by the coordinator. These are implementation records, including their historical restrictions.

A.1 Shared-Stack Proposer

The proposer receives the frozen program, running log, failure attribution, descriptor data, and patch-building interface. Its output is one shared-module change, with diagnosis and offline evidence preceding simulation. The nine builder module labels are `motion_preconditions`, `grasp_geometry`, `hold_contract`, `place_primitive`, `verifier`, `binding`, `language_adapter`, `manipulation_primitive`, and `task_driver`. These labels support attribution and dependency checks across the four conceptual layers.

Shared-Stack Proposer

Role. You are the proposer in a recursive self-improvement loop. Repair one reusable component of a shared robot program from source-task failures.
Evidence. You receive the following message:

- Frozen parent program
- Development failure attribution
- Recorded public observations
- Descriptor data
- Patch interface

Simulator truth may be used to diagnose failures and evaluate offline replay, but must never enter the executable policy.
Procedure. Choose one failure class that a coherent shared change can address across the development tasks. Identify the first violated requirement and the responsible module. Write the diagnosis before the patch. Replay the parent and candidate decisions on the recorded observations, and check that previously passing decisions remain unchanged. Derive new constants from measured geometry or documented robot quantities.
Constraints. Produce one task-agnostic patch to one declared module. Declare all affected dependencies. Use public robot and perception APIs. Read task facts from descriptors rather than embedding task, object, scene, color, or seed identifiers. Do not run policy episodes or tune constants to episode outcomes. Write only within the assigned proposal workspace.
Deliverables. Return the diagnosis, patch, offline replay and its results, and meaningful tests. The final report must state the failure class, modified module, proposed mechanism, evidence, test results, unresolved limitations, and artifact locations. The evaluator builds the candidate only after this report.

A.2 Verifier

The "verifier" role is implemented by deterministic checks and an evaluator review. The coordinator runbook specifies how to invoke the builder, coverage gate, paired rollout gate, and review, and when the accepted parent may change. The adapted instructions below summarize the coordinator's evaluation responsibilities.

Candidate validation

Role. You are the coordinator and evaluator. Control candidate evaluation, and acceptance while keeping the current parent frozen.
Proposal. Predeclare the parent, missing pairs, and development tasks. Provide only development evidence to one proposer. Resume an interrupted proposal rather than generating a duplicate. Wait for its diagnosis, replay, tests, and final report.
Validation. Build the final patch through the fixed builder and run the coverage check. A build or coverage failure consumes the proposal slot and stops candidate evaluation. Otherwise, freeze the candidate and evaluation inputs, then run the predeclared paired plan. Check complete episode accounting, policy, matched initial-state identities, and runtime integrity.
Review. Apply the capability and regression gates. Inspect new successes for genuine physical completion and inspect new regression failures. Numeric passage alone is insufficient. Do not disclose selection or hidden regression details to the proposer.
Decision. Record acceptance or rejection and its evidence. Update the parent only after all required checks and evaluator review pass. On rejection, retain the parent. At the declared stopping point, snapshot the last accepted program and record its lineage.

Check implementation. The builder checks exact patch anchors, the declared module, a forbidden-literal list in inserted text, Python compilation, and supplied `test_*.py` files. Missing tests produce a warning in the inherited builder; the protocol and evaluator require them.

Paired acceptance. Each rollout stage contains `NUM` rows: `NUM` development episodes (seeds 86–95), `NUM` selection episodes (the same tasks, seeds 76–85), `NUM` ordinary-source regression episodes, and `NUM` synthetic-regression episodes. Here "synthetic" denotes instantiated simulator tasks in the protocol. A group's net official gain must be at least two development states and one selection state. Losses are counted separately: at most one state per task per role, and at most one in each regression role. Both stages must pass integrity checks; at most four clean horizon timeouts are permitted. Passing the numeric gate is insufficient: the evaluator must inspect new physical completions and new regression failures.

B Inference Pipeline

Inference executes a frozen shared program. The data-only adapter `adapter_v3.py` constructs descriptors from the public runtime instruction `env.handle.task_language`. To keep general, it does not take a task identifier, scene identifier, seed, or simulator state as input. The program's `la_parse` produces ordered subgoals and `la_dispatch` checks every subgoal before any motion. Unsupported instructions stop explicitly. These functions are deterministic code. The following boxes present concise versions of its localization and identity prompts, using symbolic placeholders for runtime values.

B.1 Object Localization

Revision `NUM` preserves a confident text-based binding and uses VLM localization when text binding fails. The model sees a public camera image, the instruction, the current object phrase, and the execution phase. It returns a pixel or abstains. The point prompts SAM3; masks must contain the point, occupy no more

than 8% of the image, and yield at least 50 deprojected depth points. Only `medium` or `high` confidence proceeds.

Object localization | Initial reference

Context. Task instruction: `{instruction}`. Object phrase: `{phrase}`. Execution phase: `{phase}`.
Image. Image 1 is the current camera view, `{W}` by `{H}` pixels. Coordinates start at the upper left: x indexes columns and y indexes rows.
Task. Identify the single object instance referred to by the instruction, using its spatial qualifiers. Return a pixel at the center of its visible body. Abstain if the instance is ambiguous or not visible.
Response. Return abstain, x, y, confidence, and a brief reason.

When association with the frozen reference fails before grasping, the query includes the initial image with the reference highlighted in magenta. The dimensions `{W}` and `{H}`, instruction, phrase, and phase are runtime substitutions.

Object localization | Reference recovery

Context. Task instruction: `{instruction}`. Object phrase: `{phrase}`. Execution phase: `{phase}`.
Images. Image 1 is the current camera view, `{W}` by `{H}` pixels. Image 2 is the initial view with the reference instance highlighted in magenta. Coordinates in image 1 start at the upper left.
Task. Find the same physical instance in image 1, allowing for objects moved by the robot. Similar-looking objects are different instances. Return a pixel at the center of the visible body. Abstain if the reference cannot be identified confidently or is not visible.
Response. Return abstain, x, y, confidence, and a brief reason.

B.2 Choosing a Whole-Object Mask

SAM3 can return nested masks at one point. At initial binding, a separate query chooses the complete object rather than an inner surface, lid, merged support, or rival instance. Image 1 provides the full view; subsequent images show the highlighted candidate crops. The response selects one candidate image label or abstains. Reference-conditioned localization instead selects among nested masks using the frozen body's size.

Mask selection | Whole-object instance

Context. Task instruction: `{instruction}`. Object phrase: `{phrase}`. Execution phase: `{phase}`.
Images. Image 1 shows the full view with a magenta cross on the proposed object. Each subsequent image shows the same cropped region with one candidate mask highlighted.
Task. Choose the image highlighting exactly the complete requested object instance. Exclude masks covering only a label, lid, or inner surface; masks merged with a neighboring object or support; and masks of another instance.
Response. Return the candidate image label as choice and a brief reason. Return none if no candidate is valid.

B.3 Identity Before Grasping and Transport

The stack freezes object references before any object moves. Before grasping, it associates a point-prompt mask by majority overlap or a 3 cm center-distance bound; failed association invokes reference-conditioned

localization. A confirmed, unchanged reference can bypass another identity query. Otherwise, the cross-frame prompt below returns `yes`, `no`, or `unsure`.

Identity verification | Before grasping

Context. Task instruction: `{instruction}`. Object phrase: `{phrase}`. Execution phase: `{phase}`.
Images. Image 1 highlights the instance identified at task start. Image 2 highlights the current candidate.
Task. Determine whether both highlighted regions refer to the same physical instance. Account for robot-induced motion and use surrounding unmoved objects as context. Similar appearance alone does not establish identity.
Response. Return verdict as yes, no, or unsure, with a brief reason.

After grasping, the query compares the frozen reference, the pregrasp confirmation, and the current lifted object. The template accommodates either two or three images: the pregrasp confirmation is included when available.

Identity verification | After grasping

Context. Task instruction: `{instruction}`. Object phrase: `{phrase}`. Execution phase: `{phase}`.
Images. Image 1 highlights the initial reference. When available, image 2 highlights the pregrasp confirmation. The final image shows the object currently lifted between the fingers.
Task. Determine whether the lifted object is the same physical instance as the highlighted reference. Use the earlier arrangement, the grasped location, and any newly empty location as evidence. Abstain from a positive identity claim when the evidence is insufficient.
Response. Return verdict as yes, no, or unsure, with a brief reason.

Transport requires verified identity, a verified hold, a reached lift, and at least 5 cm of observed object-base rise. A positive model answer alone is insufficient.

B.4 Structured Responses and Runtime Configuration

The point-query schema requires `abstain`, integer `x`/`y`, a confidence enum, and a reason. Identity queries require a verdict enum and a reason; mask-choice queries require `choice` (a candidate image label or `none`) and a reason. Table 3 summarizes the response fields; additional fields are disallowed.

Table 3: Structured outputs for the inference prompts. Every response includes a brief reason.

Query	Required fields	Allowed values
Point localization	`abstain`, `x`, `y`, `confidence`, `reason`	Boolean abstention; integer pixel coordinates; confidence: `low`, `medium`, or `high`.
Mask selection	`choice`, `reason`	A candidate image label, or `none`.
Identity / mask review	`verdict`, `reason`	`yes`, `no`, or `unsure`.

We set the model identifier to `claude-opus-5-5`, effort to `low`, and the default response limit to 3,000 tokens. It sends images and prompt text as one user message with a JSON-schema output format; it adds no separate system prompt. Errors, refusals, truncation, or budget exhaustion return no usable answer and are handled conservatively by the policy.

Execution. Motion-dependent actions use explicit arrival/contact results, and failed transport or release checks invoke bounded recovery. The placement verifier distinguishes `ready_to_release` from `placed` and measures support geometry, identity, gripper clearance, and two-frame stability. However, this snapshot sets `enforce=False` and `corrections=False`: the placement verifier is a shadow monitor, whereas the contact-aware descent gate controls release. At the outer plan level, `run()` logs a subgoal's return value and starts the next subgoal; it does not require the shadow placement verdict to be `success`. Consequently, the implementation should not be described as enforcing every placement postcondition at every skill boundary. Official task completion is measured by the evaluator separately from controller logs.

C Implementation Details

The final artifact should include the shared-stack snapshots, source and target adapters, accepted and rejected patch ledger, evaluator implementation, exact model identifiers, environment revisions, and commands reproducing each table. Implementation records are organized by frozen program version and evaluation phase. A version record links the parent and candidate code hashes to the proposed patch, tests, gate decision, and rollout directories. Episode records identify the instruction, perturbation axis, task, seed and initial-state index, program hash, official outcome, termination status, and wall-clock duration. The inference comparison uses the N13 stack and the N13O/N13C driver records; these artifact identifiers name recorded implementations rather than additional learned stacks. A public artifact URL is not yet specified.

C.1 Grasping-backend Mismatch

Our simulation experiments use the open-source Contact-GraspNet backend rather than the AnyGraspFang et al. (2023) backend used in the ASPIRE configuration considered for comparison. This mismatch limits the fairness of a direct performance comparison, because differences in grasp proposal quality may affect both individual manipulation outcomes and long-horizon task completion. We speculate that replacing Contact-GraspNet (Sundermeyer et al., 2021) with AnyGrasp could improve grasp-dependent execution; this effect has not been measured in a matched backend comparison.

C.2 Adapter and Contract Schemas

The data-only adapter constructs descriptors from the public runtime instruction. Its fields separate `objects`, `supports`, and `articulated` parts, together with side-word mappings, language templates, anaphora templates, and support prompts. Object and support entries map referring phrases to perception prompts; they do not contain controller code. The adapter takes no benchmark task ID, seed, or simulator state as a policy input. A shared parser produces ordered subgoals, and dispatch checks whether each requested operation is supported before motion begins.

Placement distinguishes support relations from relative placement, including `on_support` and `beside_support`; later extensions add front/middle reference semantics. The supported vocabulary is versioned with the stack, and unsupported relations are refused rather than silently approximated. Dependencies are expressed by subgoal order and executable readiness checks: completion of one subgoal does not bypass the next skill's preconditions. For example, placing pudding beside a plate in a source instruction and placing it beside a plate after a mug-placement subgoal invoke the same relation-aware implementation with different runtime descriptors.

Motion results expose a status, detail, reason, reached flag, and elapsed `wall_s`. Recorded statuses include `reached`, `unreachable`, `stalled`, `uncertain`, `invalid_input`, and `budget_exhausted`. A reached motion is distinct from an established placement relation. Motion budgets are measured in control

steps; episode limits additionally use wall-clock seconds. In the recorded inference-driver comparison, the common limits are 4,000 simulation steps and 1,500 seconds per episode, including model calls.

C.3 Regression Gates

Candidate construction checks exact patch anchors, the declared shared module, forbidden literals in inserted code, Python compilation, and supplied test files. Identifier checks constrain explicit specialization to benchmark tasks or seeds. These automated checks do not establish semantic generality; review additionally examines whether geometric constants follow calibration, measured geometry, or an object-independent construction. Missing tests produce a warning in the inherited builder, while the evaluation protocol requires supporting tests before acceptance.

Mechanism tests exercise the changed behavior and its failure returns. Recorded observations support replay of perception and verification decisions; changes to motion require execution evidence. Parent and candidate are then run from matched source initial states. The gate configuration is fixed at the beginning of each RSI iteration and specifies the evaluation stages, development and selection panels, regression tasks, required improvements, paired-loss limits, and integrity and timeout checks. These settings can evolve across iterations, but are held fixed for all candidate comparisons within a given iteration. The rollout manifest records the corresponding seed-to-state mapping and resource budgets.

Numeric acceptance is followed by inspection of newly successful trajectories and new regression failures. The parent changes only after the checks and review required by the current iteration pass; otherwise the frozen parent is retained. Selection and regression outcomes used by this gate are acceptance evidence, not an independent audit. Any audit panel must be excluded from proposal generation and candidate selection, and reported separately. Target episodes do not supply evidence to the source acceptance gate.

C.4 Source–Target Combination Generalization

Source tasks provide manipulation constituents such as grasping, transport, support placement, relative placement, and articulated-object actuation. Target instructions combine these constituents into longer goal sequences. Representative compositions include stove activation followed by pan placement, mug placement followed by pudding placement beside a plate, and bowl containment followed by drawer closure. Object phrases specify the manipulated object and reference; relation types distinguish the required goal; ordered subgoals specify the composition. Reuse is therefore evaluated at the level of shared executable calls rather than instruction-string overlap alone.

The main LIBERO-Pro Long protocol comprises ten tasks with 50 states per task and axis. The separate inference-driver comparison uses five states per task and axis, seeds 41–45, for 100 episodes. These panels have different denominators and are not pooled. Pairing uses task, suite, seed, and recorded initial-state index, with the benchmark revision needed to reconstruct the state. A seed label alone is insufficient to establish identical initial conditions across revisions.

Frozen execution excludes code modification during the target episode. It does not by itself establish that evaluation tasks were never observed during development. The recorded inference comparison reused evaluation rows, and its protocol identifies historical exposure to Scene5. These records must be distinguished from a newly isolated target evaluation when interpreting compositional generalization.

C.5 Resource Accounting

Official full-task outcomes are retained for every planned evaluation row, including failures, timeouts, generation refusals, and execution errors. Success is aggregated per task and axis before macro-averaging. Paired comparisons additionally count transitions from parent failure to candidate success and from parent

success to candidate failure, since net success can conceal lost capabilities. The reported exact McNemar comparisons use these discordant outcomes. Confidence intervals are not inferred from rounded aggregate rates; they require episode outcomes and a procedure that respects task grouping.

Source improvement and target inference are accounted for separately. Source records distinguish proposal generation, offline tests, candidate rollouts, and rejected candidates. Inference records distinguish initial program generation, runtime decision and perception calls, and robot execution. End-to-end episode time includes generation and execution; model calls inside execution are not added a second time. The figure reports mean duration over all episodes, not only successes. Internal bounded retries remain part of their original episode and consume its budget.

For N13C, the reported amended comparison substitutes five infrastructure-failed rows with their designated reruns; the original records remain available. Smoke episodes and offline generation tests are excluded from the 100 policy outcomes, while their costs remain separate development expenditures. Generated-tool counts should distinguish newly registered operations from revisions to existing ones; invocation traces establish use, whereas matched removal tests are needed to attribute a success gain to a component. No total cost, repeat count, or tool count is inferred where the corresponding ledger is absent.

D LIBERO-Pro Long Zero-Shot Evaluation Details

D.1 Task Manifest and Frozen Execution

The per-task breakdown uses the 10 LIBERO-Pro Long tasks and the *Pos*/*Task* variants listed in ASPIRE's zero-shot appendix (Lu et al., 2026). Descriptive task names below follow that appendix; the artifact must map each name to its exact benchmark task ID, instruction, and initial-state manifest. Each task–axis pair uses seeds 1–50 and a fixed task program. All shared code and adapters are finalized before these episodes; target feedback cannot select alternative programs or patches. The artifact records the source task order, final stack hash, generated program and adapter hashes, benchmark revision, seed-to-state mapping, execution horizon, and recovery allowance.

D.2 Source-Experience Scaling

The scaling evaluation in Figure 7 uses one-shot task-program inference at every checkpoint. A task program is generated once and executed over the frozen shared stack without target-side code debugging; its execution may nevertheless use observation feedback. Each plotted value is from a single evaluation run, rather than an average across repeated runs. Figure 7 reports the source-experience scaling evaluation for frozen RECURSIA stacks after $N \in \{0, 3, 6, 9, 10, 12, 13, 14, 15, 16, 17, 18\}$ improvement iterations. Each snapshot is selected using source data only and evaluated without target-driven modification.

D.3 Aggregation

With $y_{ias} \in \{0, 1\}$ denoting official full-task success on seed s, compute

$$\widehat{p}_{ia} = \frac{1}{50} \sum_{s=1}^{50} y_{ias}, \qquad \widehat{p}_{a} = \frac{1}{10} \sum_{i=1}^{10} \widehat{p}_{ia}, \qquad \widehat{p}_{\text{Overall}} = \frac{\widehat{p}_{\text{Pos}} + \widehat{p}_{\text{Task}}}{2}. \tag{7}$$

Our aggregate results are computed from unrounded episode outcomes. Reproduced baseline aggregates retain the source's reported precision. Confidence intervals must account for the grouping of states within tasks, and paired comparisons require matched episode-level outcomes.

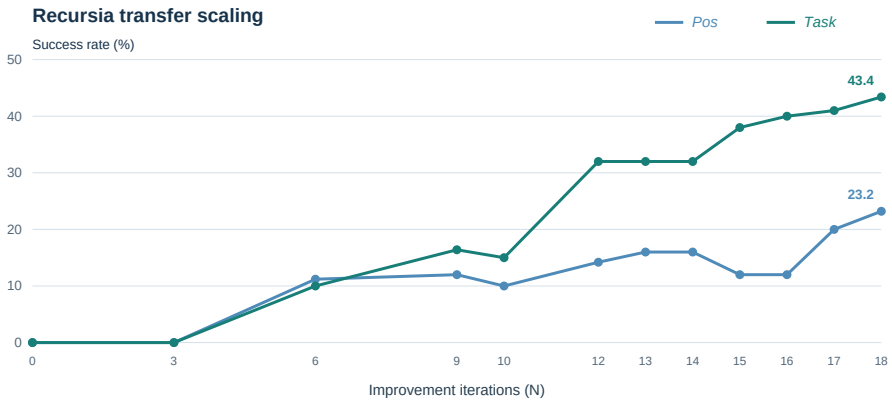


Figure 7: RECURSIA transfer scaling with source-task experience. Frozen-stack success rates on *Pos* and *Task* are shown at the supplied iteration checkpoints.